Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИсп1
Зонова А.М.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1.Постановка задачи 4
2.Руководство пользователя 6
3.Руководство программиста 11
- Описание структуры программы 11
- Описание структур данных 11
- Описание алгоритмов 11

4.Результаты экспериментов и тестирования 14
Заключение 16
Литература 17
Приложение 18

Введение

Битовое поле — это структура данных, которая соответствует одному или нескольким соседним битам, выделенным для определенных целей, так что любой отдельный бит или группа битов в структуре могут быть установлены или проверены. Битовое поле чаще всего используется для представления целочисленных типов с известной фиксированной разрядностью, например однобитовых булевых значений.

Значение отдельных битов в поле определяется программистом. Например, первый бит в битовом поле (расположенный по базовому адресу поля) иногда используется для определения состояния определенного атрибута, связанного с битовым полем.

Битовые поля можно использовать для снижения потребления памяти, когда программе требуется несколько целочисленных переменных, которые всегда будут иметь низкие значения. Например, во многих системах для хранения целочисленного значения требуется два байта (16 бит) памяти; иногда для хранения значений достаточно одного или двух битов. Использование нескольких таких крошечных переменных в одном битовом поле позволяет эффективно упаковывать данные в памяти.

Битовое множество — это абстрактная структура данных, которая хранит набор элементов с помощью битового вектора (массива битов). Идея в том, чтобы каждому элементу множества присвоить уникальный номер (индекс), а затем проверять или устанавливать соответствующий бит: если бит установлен — элемент принадлежит множеству, если сброшен — нет.

Цель работы: разработка структуры данных для хранения множеств с использованием битовых полей и освоение Google Test для автоматического тестирования.

1. Постановка задачи

1) Реализация класса битового поля TBitField

Необходимо разработать класс для компактного хранения и обработки последовательности битов произвольной длины. Управление памятью должно осуществляться динамически с помощью массива элементов типа TELEM (unsigned int).

Интерфейс:

1. Конструкторы и деструктор:

Конструктор, принимающий длину поля в битах, выделяющий динамическую память под массив и инициализирующий все биты нулями. Длина в машинных словах должна вычисляться с учетом остатка от деления.

Конструктор копирования, обеспечивающий глубокое копирование (выделение новой памяти и копирование содержимого).

Деструктор, освобождающий динамически выделенную память.

2. Служебные методы:

GetMemIndex(n) – вычисление индекса элемента массива, содержащего n-й бит.

GetMemMask(n) – формирование битовой маски для доступа к n-му биту внутри элемента массива.

GetLength() – возвращение общей длины битового поля в битах.

3. Операции над битами:

SetBit(n) – установка n-го бита в 1.

ClrBit(n) – сброс n-го бита в 0.

GetBit(n) – возвращение значения n-го бита.

4. Перегрузка операторов:

Сравнения: == и !=.

Присваивания: = (с обязательной проверкой на самоприсваивание).

Побитовые: | (или), & (и), ~ (инверсия).

5. Потоковый ввод-вывод:

operator>> – посимвольный ввод '0' и '1'. При вводе недопустимого символа должно генерироваться исключение std::invalid_argument.

operator<< – вывод последовательности '0' и '1', соответствующей состоянию поля.

2) Реализация класса множества TSet

Необходимо разработать класс, реализующий абстракцию математического множества целых неотрицательных чисел, используя класс TBitField в качестве внутреннего представления.

Интерфейс:

1. Конструкторы:

Конструктор, принимающий максимальную мощность множества, инициализирующий внутреннее битовое поле соответствующей длины.

Конструктор копирования.

Конструктор преобразования типа из объекта TBitField.

2. Методы доступа и модификации:

GetMaxPower() – возвращение максимальной мощности множества.

InsElem(Elem) – добавление элемента в множество.

DelElem(Elem) – удаление элемента из множества.

IsMember(Elem) – проверка принадлежности элемента множеству.

3. Перегрузка операторов:

Сравнения: == и !=.

Присваивания: = (с проверкой на самоприсваивание).

Теоретико-множественные операции, возвращающие новый объект TSet:

- + с целым числом: возвращает копию множества с добавленным элементом.
- - с целым числом: возвращает копию множества с удаленным элементом.
- + с объектом TSet: объединение множеств.
- * с объектом TSet: пересечение множеств.

- ~ (дополнение): важная особенность реализации – инверсия должна затрагивать только биты в пределах MaxPower, чтобы избежать появления "мусорных" единиц из-за неполного заполнения последнего машинного слова.

Оператор преобразования типа к TBitField.

4. Потоковый ввод-вывод:

operator>> – ввод элементов множества до встречи с отрицательным числом (сигнал конца ввода). Должны генерироваться исключения: std::out_of_range, если элемент ≥ MaxPower, и std::invalid_argument при некорректном формате.

operator<< – форматированный вывод элементов множества в виде {a, b, c} с корректной расстановкой запятых.

3) Тестирование

Необходимо разработать набор автоматизированных тестов с использованием Google Test, покрывающих основной функционал и проверяющих устойчивость программы к некорректным данным.

Требования к покрытию тестами:

1. Тестирование TBitField:

Корректность выделения памяти и начальной инициализации нулями.

Работа методов SetBit, ClrBit, GetBit на границах машинных слов.

Корректность глубокого копирования и оператора присваивания (включая проверку самоприсваивания bf = bf).

Логика побитовых операций |, &, ~ для полей разной и одинаковой длины.

2. Тестирование TSet:

Добавление, удаление и проверка элементов.

Корректность операций +, -, *, ~ (особенно проверка, что оператор ~ не добавляет лишние элементы за пределы MaxPower).

Преобразование типов между TSet и TBitField.

3. Тестирование исключительных ситуаций и краевых случаев:

Генерация std::invalid_argument при попытке ввода нечислового символа в множество.

Генерация std::out_of_range при попытке добавить элемент, индекс которого ≥ MaxPower.

Корректная остановка ввода при считывании отрицательного числа (отрицательное число не должно добавляться в множество и должно завершать цикл ввода).

2. Руководство пользователя

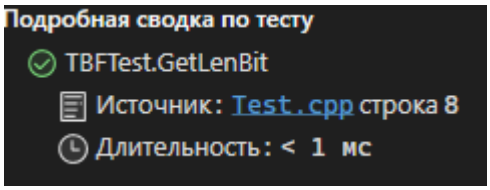
Программа представляет собой библиотеку классов на языке C++ для работы с множествами целых неотрицательных чисел на основе битовых полей. Основные компоненты:

- **TBitField** — низкоуровневый класс для хранения и побитовой обработки данных.
- **TSet** — высокоуровневый класс, реализующий абстракцию математического множества поверх TBitField.

Программа предназначена для автоматизированного тестирования с помощью фреймворка Google Test. Интерактивный ввод с клавиатуры не требуется.

Программное использование класса TBitField.
Тесты являются наглядным примером использования класса TBitField, если тест запускается без ошибок, тот или иной метод реализован верно, поэтому следующие методы пользователь проверяет на тестах:
Создание и инициализация битового поля:

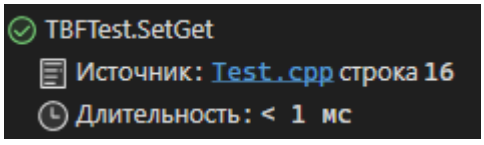
```
TEST(TBFTest, GetLenBit) { //проверка,
    TBitField bf(1000);
    EXPECT_EQ(bf.GetLength(), 1000);
    for (int i = 0; i < 1000; i++) {
        EXPECT_EQ(bf.GetBit(i), 0);
    }
}
```



Конструктор TBitField(int len) корректно выделяет память под 1000 бит.
Метод GetLength() возвращает точное количество бит.
Все биты после создания равны 0.
Установка и получение значений битов:

```
TEST(TBFTest, SetGet) { //проверка,
    TBitField bf(50);
    bf.SetBit(0);
    bf.SetBit(4);

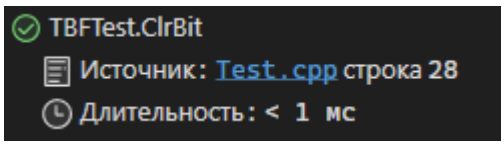
    EXPECT_EQ(bf.GetBit(0), 1);
    EXPECT_EQ(bf.GetBit(4), 1);
    EXPECT_EQ(bf.GetBit(1), 0);
    EXPECT_EQ(bf.GetBit(2), 0);
    EXPECT_EQ(bf.GetBit(49), 0);
}
```



Метод SetBit(n) устанавливает n-й бит в 1, не затрагивая соседние.
Метод GetBit(n) корректно возвращает 1 для установленных битов и 0 для остальных.
Работа с битами на границах поля (бит 0 и бит 49).
Сброс битов:

```
TEST(TBFTest, ClrBit) { //очищение би
    TBitField bf(5);
    bf.SetBit(3);
    EXPECT_EQ(bf.GetBit(3), 1);
    bf.SetBit(4);
    bf.ClrBit(3);

    EXPECT_EQ(bf.GetBit(3), 0);
    EXPECT_EQ(bf.GetBit(4), 1);
}
```

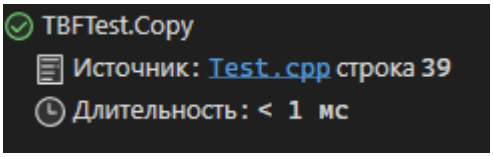


Метод ClrBit(n) сбрасывает только указанный бит, не влияя на остальные.
Конструктор копирования:

```
TEST(TBFTest, Copy) { //конструктор копиро
    TBitField bf1(5);
    bf1.SetBit(2);
    EXPECT_EQ(bf1.GetBit(2), 1);

    TBitField bf2(bf1); //копирование

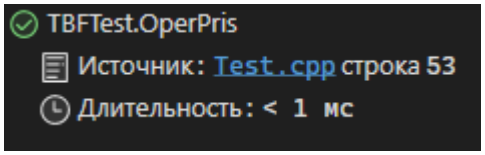
    EXPECT_EQ(bf2.GetBit(2), 1);
    EXPECT_EQ(bf2.GetBit(4), 0);
    bf2.SetBit(4);
    EXPECT_EQ(bf2.GetBit(4), 1);
    EXPECT_EQ(bf1.GetBit(4), 0);
}
```



Конструктор копирования TBitField(const TBitField& bf) создает независимую копию.
Изменение копии не влияет на оригинал — память выделена отдельно.
Оператор присваивания:

```
TEST(TBFTest, OperPris) { //проверка
    TBitField bf1(5);
    bf1.SetBit(4);
    EXPECT_EQ(bf1.GetBit(4), 1);
    TBitField bf2(5);
    bf2 = bf1;
    EXPECT_EQ(bf2.GetBit(4), 1);

    bf1 = bf1;
    EXPECT_EQ(bf1.GetBit(4), 1);
}
```



Оператор = корректно копирует содержимое одного поля в другое.
Случай самоприсваивания (bf1 = bf1) обрабатывается корректно — данные не теряются благодаря проверке.
Побитовые операции:

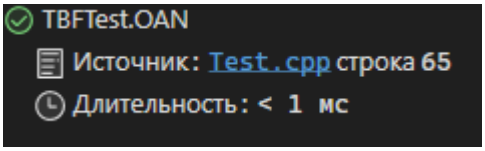
```
TEST(TBFTest, OAN) { //битовые операции
    TBitField bf1(5);
    bf1.SetBit(1);
    bf1.SetBit(2); // 0110

    TBitField bf2(5);
    bf2.SetBit(2);
    bf2.SetBit(3); // 1100

    //объединение
    TBitField bf_or = bf1 | bf2;
    EXPECT_EQ(bf_or.GetBit(0), 0);
    EXPECT_EQ(bf_or.GetBit(1), 1);
    EXPECT_EQ(bf_or.GetBit(2), 1);
    EXPECT_EQ(bf_or.GetBit(3), 1);
    EXPECT_EQ(bf_or.GetBit(4), 0);

    //пересечение
    TBitField bf_and = bf1 & bf2;
    EXPECT_EQ(bf_and.GetBit(0), 0);
    EXPECT_EQ(bf_and.GetBit(1), 0);
    EXPECT_EQ(bf_and.GetBit(2), 1);
    EXPECT_EQ(bf_and.GetBit(3), 0);
    EXPECT_EQ(bf_and.GetBit(4), 0);

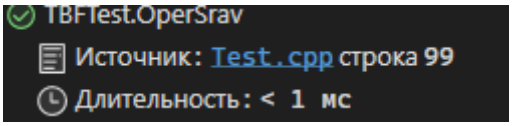
    //инверсия
    TBitField bf_not = ~bf1;
    EXPECT_EQ(bf_not.GetBit(0), 1);
    EXPECT_EQ(bf_not.GetBit(1), 0);
    EXPECT_EQ(bf_not.GetBit(2), 0);
    EXPECT_EQ(bf_not.GetBit(3), 1);
    EXPECT_EQ(bf_not.GetBit(4), 1);
}
```



Оператор | выполняет побитовое *или* — объединяет установленные биты обоих операндов.
Оператор & выполняет побитовое *и* — оставляет только общие установленные биты.
Оператор ~ инвертирует все биты поля.
Все операции возвращают новый объект, не модифицируя исходные операнды.
Операторы сравнения:

```
TEST(TBFTest, OperSrav) { //опера
    TBitField bf1(5);
    bf1.SetBit(4);
    TBitField bf2(5);
    bf2.SetBit(4);
    TBitField bf3(5);
    bf3.SetBit(3);

    EXPECT_TRUE(bf1 == bf2);
    EXPECT_FALSE(bf1 == bf3);
    EXPECT_TRUE(bf1 != bf3);
}
```



Оператор == возвращает истину, если длины полей совпадают и все биты равны.
Оператор != работает как логическое отрицание ==.

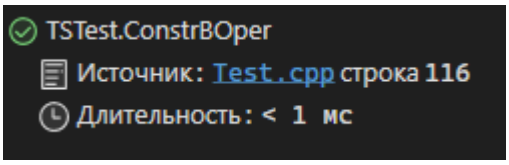
Программное использование класса TSet.
Создание множества и базовые операции с элементами:

```
TEST(TSTest, ConstrBOper) { //провер
    TSet s(5);
    EXPECT_EQ(s.GetMaxPower(), 5); /

    s.InsElem(3); //добавить элемент
    s.InsElem(4);

    EXPECT_TRUE(s.IsMember(3)); //на
    EXPECT_TRUE(s.IsMember(4));
    EXPECT_FALSE(s.IsMember(2));

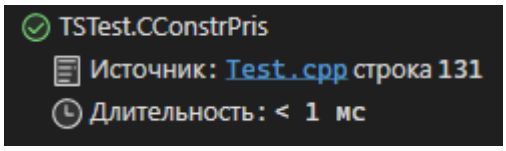
    s.DelElem(3); //удалить элемнт
    EXPECT_FALSE(s.IsMember(3));
}
```



Конструктор TSet(int mp) задает максимальную мощность множества.
Методы InsElem и DelElem изменяют состояние множества.
Метод IsMember возвращает true/false в зависимости от принадлежности элемента.
Конструктор копирования и оператор присваивания:

```
TEST(TSTest, CConstrPris) { //провер
    TSet s1(5);
    s1.InsElem(2);
    EXPECT_TRUE(s1.IsMember(2));
    TSet s2(s1);
    EXPECT_TRUE(s2.IsMember(2));
    s2.InsElem(3);
    EXPECT_FALSE(s1.IsMember(3));

    TSet s3(5);
    s3 = s1;
    EXPECT_TRUE(s3.IsMember(2));
}
```

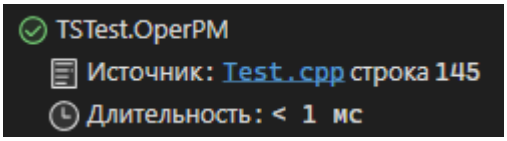


Глубокое копирование: внутреннее битовое поле копируется полностью, объекты независимы.
Оператор = корректно переопределен и обрабатывает присваивание.
Операции с отдельными элементами:

```
TEST(TSTest, OperPM) { //добавить, удали
    TSet s(6);
    s.InsElem(1);

    TSet s_plus = s + 5;
    EXPECT_TRUE(s_plus.IsMember(1));
    EXPECT_TRUE(s_plus.IsMember(5));
    EXPECT_FALSE(s.IsMember(5));

    TSet s_minus = s_plus - 1;
    EXPECT_FALSE(s_minus.IsMember(1));
    EXPECT_TRUE(s_minus.IsMember(5));
    EXPECT_TRUE(s_plus.IsMember(1));
}
```



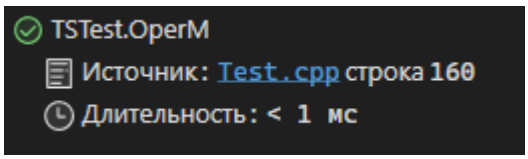
Операторы + int и - int не модифицируют исходное множество, а возвращают новый объект TSet.
Это соответствует принципам функционального стиля и безопасности данных.
Теоретико-множественные операции:

```
TEST(TSTest, OperM) { //теоретико-множественны
    TSet A(10);
    A.InsElem(1);
    A.InsElem(2);
    TSet B(10);
    B.InsElem(2);
    B.InsElem(3);

    //объединение
    TSet C = A + B;
    EXPECT_TRUE(C.IsMember(1));
    EXPECT_TRUE(C.IsMember(2));
    EXPECT_TRUE(C.IsMember(3));

    //пересечение
    TSet D = A * B;
    EXPECT_TRUE(D.IsMember(2));
    EXPECT_FALSE(D.IsMember(1));
    EXPECT_FALSE(D.IsMember(3));

    //дополнение
    TSet E(5);
    E.InsElem(0);
    E.InsElem(2); // {0, 2}
    TSet otr = ~E; // Должно быть {1, 3, 4}
    EXPECT_FALSE(otr.IsMember(0));
    EXPECT_TRUE(otr.IsMember(1));
    EXPECT_FALSE(otr.IsMember(2));
    EXPECT_TRUE(otr.IsMember(3));
    EXPECT_TRUE(otr.IsMember(4));
}
```

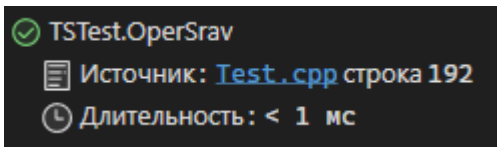


+ между множествами выполняет объединение.
* между множествами выполняет пересечение.
~ выполняет дополнение, инвертирует биты только в пределах MaxPower, что предотвращает появление лишних элементов из-за неполного заполнения последнего машинного слова битового поля.

Операторы сравнения:

```
TEST(TSTest, OperSrav) { //опера
    TSet s1(10);
    s1.InsElem(5);
    TSet s2(10);
    s2.InsElem(5);
    TSet s3(10);
    s3.InsElem(6);

    EXPECT_TRUE(s1 == s2);
    EXPECT_FALSE(s1 == s3);
    EXPECT_TRUE(s2 != s3);
}
```



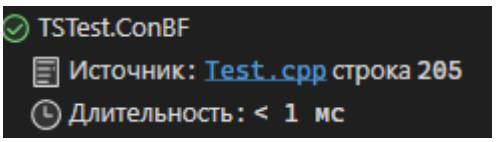
Сравнение множеств делегируется сравнению их внутренних битовых полей. Множества равны, если совпадают их мощности и наборы элементов.

Преобразование типов между TSet и TBitField:

```
TEST(TSTest, ConBF) { //преобразовани
    TBitField bf(10);
    bf.SetBit(4);
    bf.SetBit(8);

    //конструктор из битового поля
    TSet s(bf);
    EXPECT_TRUE(s.IsMember(4));
    EXPECT_TRUE(s.IsMember(8));
    EXPECT_EQ(s.GetMaxPower(), 10);

    //оператор преобразования в битов
    TBitField bf2 = (TBitField)s;
    EXPECT_EQ(bf2.GetBit(4), 1);
    EXPECT_EQ(bf2.GetBit(8), 1);
}
```

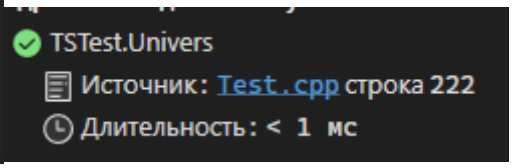


Прямое использование TBitField для инициализации TSet.

Явное приведение типа (TBitField)set для получения низкоуровневого представления множества.

```
TEST(TSTest, Univers) { //проверка для разных универсов
    TSet A(10);
    TSet B(20);
    A.InsElem(5);
    B.InsElem(15);

    EXPECT_THROW(A + B, std::invalid_argument);
    EXPECT_THROW(A * B, std::invalid_argument);
    EXPECT_THROW(A == B, std::invalid_argument);
    EXPECT_THROW(A != B, std::invalid_argument);
}
```



Операции объединения, пересечения и сравнения могут совершаться только, если совпадают их мощности, поэтому необходимо добавить исключение, которое выбрасывается при не равных мощностях множеств.

Обработка исключительных ситуаций

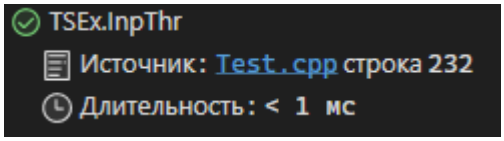
При программном использовании или чтении из потоков классы генерируют стандартные исключения C++ при нарушении условий корректности:

std::out_of_range

Генерируется при попытке добавить или проверить элемент, индекс которого больше или равен максимальной мощности (MaxPower) множества.

```
TEST(TSEx, InpThr) { //выход за пределы мощности
    TSet s(5);
    std::istringstream bad("1 3 6 -1");

    EXPECT_THROW(bad >> s, std::out_of_range);
}
```

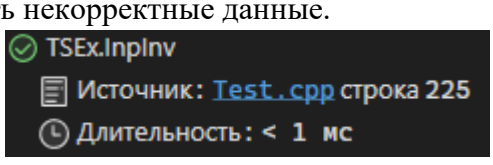


std::invalid_argument

Генерируется при попытке прочитать некорректные данные.

```
TEST(TSEx, InpInv) { //ввод недопустимого значения
    TSet s(10);
    std::istringstream bad("1 2 A 3");

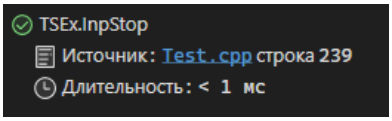
    EXPECT_THROW(bad >> s, std::invalid_argument);
}
```



Остановка ввода при встрече отрицательного числа.

```
TEST(TSEx, InpStop) { //ввод корректно останавливается при во
    TSet s(10);
    std::istringstream good("1 3 5 -1 8");
    good >> s;

    EXPECT_TRUE(s.IsMember(1));
    EXPECT_TRUE(s.IsMember(3));
    EXPECT_TRUE(s.IsMember(5));
    EXPECT_FALSE(s.IsMember(8)); // 8 не должна добавиться
}
```



Запуск автоматических тестов

Основной способ проверки работоспособности программы — запуск набора тестов Google Test, встроенного в проект.

1. Скомпилируйте проект вместе с файлами тестов.
2. Запустите исполняемый файл.
3. Фреймворк автоматически выполнит все сценарии, определенные в макросах TEST и TEST_F:
    - TBFTest: проверка корректности работы TBitField (инициализация, операции, копирование).
    - TSTest: проверка корректности работы TSet (добавление, удаление, теоретико-множественные операции).
    - TSEx: проверка корректного выброса исключений std::out_of_range и std::invalid_argument при некорректных данных.

Успешное выполнение подтверждается сообщением фреймворка:

```
[----------] Global test environment tear-down
[==========] 18 tests from 3 test cases ran. (10 ms total)
[  PASSED  ] 18 tests.
```

3.Руководство программиста

Описание структуры программы

Проект состоит из следующих модулей:

1. Модуль битового поля

Назначение: реализация низкоуровневой структуры данных для хранения и побитовой обработки последовательности битов произвольной длины.

Состав:

- Класс TBitField.
- Глобальные перегруженные операторы потокового ввода operator>> и вывода operator<< для класса TBitField.

2. Модуль множества

Назначение: реализация высокоуровневой абстракции математического множества целых неотрицательных чисел на основе класса TBitField.

Состав:

- Класс TSet.
- Глобальные перегруженные операторы потокового ввода operator>> и вывода operator<< для класса TSet.

3. Модуль тестирования

Назначение: автоматизированная проверка корректности работы классов TBitField и TSet, включая обработку граничных случаев и исключений.

Состав: Наборы тестов TBFTest (для битового поля), TSTest (для множеств) и TSEx (для исключительных ситуаций), использующие макросы фреймворка Google Test.

4. Заголовочные файлы интерфейсов (includes/Lab1/)

Назначение: хранение объявлений классов и прототипов функций. Обеспечивают возможность подключения модулей к другим частям проекта без изменения их внутренних реализаций.

Описание структур данных

В программе не используются глобальные переменные, что соответствует принципам инкапсуляции и безопасного программирования. Все данные инкапсулированы внутри классов.

1. Класс TBitField

Класс хранит массив машинных слов, каждый бит которого представляет элемент битового поля.

Поля данных (приватные):

- int BitLen — общая длина битового поля в битах.
- int MemLen — длина битового поля в машинных словах (элементах массива pMem).
- TELEM* pMem — указатель на динамически выделенный массив машинных слов (тип TELEM обычно определяется как unsigned int).

2. Класс TSet

Класс является надстройкой над TBitField и использует его для хранения элементов множества.

Поля данных (приватные):

- int MaxPower — максимальная мощность множества. Определяет максимальное значение элемента, которое может быть добавлено в множество.
- TBitField BitField — внутреннее битовое поле, в котором индекс установленного бита соответствует значению элемента множества.

Описание алгоритмов

1. Алгоритм адресации бита

Для доступа к n-му биту в массиве машинных слов используется следующий алгоритм:

1. Вычисление индекса машинного слова: номер элемента массива, в котором находится бит, вычисляется путем целочисленного деления индекса бита на количество бит в одном слове: index = n / (sizeof(TELEM) * 8) (для 32-битного unsigned int это деление на 32).

2. Вычисление битовой маски: позиция бита внутри слова определяется остатком от деления: r = n % (sizeof(TELEM) * 8). Маска формируется сдвигом единицы влево: mask = (TELEM)1 << r.

2. Алгоритм выделения памяти

При создании битового поля длиной len бит необходимо выделить массив машинных слов. Чтобы корректно округлить результат деления вверх (если длина не кратна 32), используется формула: MemLen = (len + sizeof(TELEM) * 8 - 1) / (sizeof(TELEM) * 8). После выделения памяти (new TELEM[MemLen]) весь массив инициализируется нулями.

3. Алгоритмы теоретико-множественных операций

Операции над множествами реализуются через соответствующие побитовые операции над внутренними полями BitField:

- Объединение. Результирующее поле имеет длину, равную максимуму из длин операндов. Если длины полей различаются, "лишние" биты более длинного поля копируются в результат без изменений.

- Пересечение. Результирующее поле имеет длину, равную минимуму из длин операндов (элементы, выходящие за пределы меньшего множества, не могут принадлежать пересечению).

- Разность: реализуется как пересечение первого множества с инверсией второго (в коде разность с элементом реализована через сброс конкретного бита).

- Дополнение. Прямая инверсия битового поля (~BitField) инвертирует все биты, включая неиспользуемые старшие биты последнего машинного слова, что приводит к появлению "мусорных" элементов. Алгоритм корректного дополнения:

    1. Создается пустое битовое поле bf длиной MaxPower.

    2. Создается инвертированное поле inv = ~BitField.

    3. В цикле от 0 до MaxPower - 1 проверяются биты поля inv. Если бит равен 1, он устанавливается в поле bf.

    4. Возвращается новое множество, созданное из поля bf. Это гарантирует, что в дополнении не появятся элементы за пределами универсума.

4. Алгоритм потокового ввода с обработкой исключений

- Для TBitField: считывает символы из потока по одному. Если символ '1' — бит устанавливается, если '0' — сбрасывается. Любой другой символ прерывает ввод и генерирует исключение std::invalid_argument.

- Для TSet: считывает целые числа из потока в цикле.

    1. Если число отрицательное, цикл прерывается (сигнал конца ввода).

2. Если число ≥ MaxPower, генерируется исключение std::out_of_range (выход за пределы).

3. Если считывание числа невозможно (например, встретилась буква), поток переходит в состояние fail(), и генерируется исключение std::invalid_argument.

4. Если мощности множеств различны, то при выполнении операций пересечения, объединения и сравнения генерируется исключение std::invalid_argument.

4. Результаты экспериментов и тестирования

Для проверки корректности работы разработанных классов TBitField и TSet было проведено тестирование с использованием фреймворка Google Test. Данный подход позволяет изолировать каждый класс и проверить его функциональность независимо от других компонентов системы.

Тесты разделены на три логические группы:

1. **TBFTest** — тестирование базового класса битового поля.
2. **TSTest** — тестирование класса множества и теоретико-множественных операций.
3. **TSEx** — тестирование обработки исключительных ситуаций и граничных случаев.

Описание экспериментов (сценариев тестирования)

1. Эксперименты с классом TBitField (Группа TBFTest)

В данной группе проверяется низкоуровневая работа с памятью и побитовыми операциями:

- Инициализация (GetLenBit): проверяется, что при создании поля заданной длины (например, 1000 бит) память выделяется корректно, а все биты по умолчанию инициализируются нулями.
- Работа с битами (SetGet, ClrBit): проверяется установка, сброс и считывание битов по их индексам, включая проверку того, что операции над одним битом не влияют на соседние.
- Управление памятью (Copy, OperPris): тесты убеждаются, что конструктор копирования и оператор присваивания создают глубокую копию — независимый блок памяти. Также проверяется безопасная обработка самоприсваивания.
- Побитовые операции (OAN): проверяются операторы объединение, пересечени и инверсия для полей одинаковой длины.
- Сравнение (OperSrav): проверяется корректность операторов == и !=.

2. Эксперименты с классом TSet (Группа TSTest)

В данной группе проверяется математическая корректность абстракции множества:

- Базовые операции (ConstrBOper): добавление (InsElem), удаление (DelElem) и проверка принадлежности (IsMember) элементов.
- Операции с возвратом нового объекта (OperPM): проверяется, что операторы + int и - int не меняют исходное множество, а возвращают новую копию с изменениями.
- Теоретико-множественные операции (OperM): проверка объединения, пересечения и дополнения. Особое внимание уделяется оператору дополнения: тестируется, что инверсия происходит строго в пределах заданной мощности (MaxPower), без захвата «мусорных» битов за пределами. Так же уделяем внимание случаю, что операции объединения и пересечения не происходят с множествами разных мощностей.
- Преобразование типов (ConBF): проверяется двусторонняя конвертация между TSet и TBitField.

3. Эксперименты по обработке ошибок (Группа TSEx)

Проверяется устойчивость программы к некорректным входным данным при потоковом вводе:

- Некорректный формат (InpInv): попытка ввести буквенный символ вместо числа. Ожидается выброс std::invalid_argument.
- Выход за границы (InpThr): попытка добавить элемент, индекс которого больше или равен MaxPower. Ожидается выброс std::out_of_range.
- Корректное завершение (InpStop): проверка того, что ввод элементов множества корректно прерывается при считывании отрицательного числа, и это число не добавляется в множество.

Анализ результатов тестирования

По итогам запуска фреймворка Google Test все разработанные тесты завершились успешно. Анализ пройденных тестов позволяет сделать следующие выводы о качестве и корректности реализации:

Корректность управления динамической памятью

Успешное прохождение тестов Copy и OperPris доказывает, что в классах корректно реализовано глубокое копирование. Изменение скопированного объекта не приводит к изменению оригинала, что исключает случайной порчи данных. Обработка самоприсваивания гарантирует, что объект не удалит собственную память до ее копирования.

Математическая точность операций над множествами

Тест OperM подтвердил корректность реализации оператора дополнения. Как показал анализ кода, наивная инверсия всего битового поля (~BitField) привела бы к появлению ложных элементов в множестве из-за инверсии неиспользуемых старших битов последнего блока памяти. Реализованный алгоритм с циклом до MaxPower успешно прошел тест, доказав, что дополнение множества формируется строго в рамках заданного универсума.

Изоляция ошибок

Поскольку тесты для TBitField (TBFTest) проходят успешно, мы можем с уверенностью утверждать, что любые потенциальные ошибки в работе TSet связаны исключительно с логикой самого класса TSet, а не с низкоуровневой работой с битами. Это подтверждает правильность выбранной композиции классов.

Надежность и отказоустойчивость

Тесты группы TSEx доказали, что программа не подвержена неопределенному поведению при некорректном вводе. Вместо аварийного завершения программа генерирует стандартные исключения C++ (std::out_of_range, std::invalid_argument). Это позволяет вызывающему коду перехватить ошибку и корректно обработать её, что является признаком надежного программного обеспечения.

Проведенные эксперименты подтверждают, что разработанные классы TBitField и TSet полностью соответствуют постановке задачи. Реализация обеспечивает эффективное использование памяти (за счет битовых полей), математическую корректность операций над множествами и безопасную обработку граничных случаев. Автоматические тесты покрывают как основной функционал, так и граничные случаи.

Заключение

В ходе выполнения данной лабораторной работы была достигнута основная цель — разработана структура данных для хранения и обработки множеств на основе битовых полей, а также освоен фреймворк для автоматического тестирования Google Test.

В результате проделанной работы были получены следующие результаты:

1. Реализован класс TBitField, обеспечивающий эффективное хранение и обработку последовательностей битов произвольной длины. Класс корректно управляет динамической памятью и поддерживает все необходимые побитовые операции.

2. Создан класс TSe**t**, реализующий абстракцию математического множества целых неотрицательных чисел. Класс использует механизм композиции с TBitField и поддерживает полный набор теоретико-множественных операций (объединение, пересечение, разность, дополнение), а также операции с отдельными элементами.

3. Разработан и успешно выполнен комплекс модульных тестов с использованием фреймворка Google Test. Написанные тесты полностью покрывают основной функционал классов, проверяют корректность копирования объектов, а также тестируют граничные случаи и сценарии обработки исключительных ситуаций.

В процессе реализации были решены следующие важные технические задачи:

- Обеспечена безопасная работа с динамической памятью: реализованы деструктор, конструктор копирования и оператор присваивания, что исключает утечки памяти и ошибки при глубоком копировании объектов.
- Разработан корректный алгоритм для оператора дополнения множества, который инвертирует биты строго в пределах заданной мощности (MaxPower), предотвращая появление ложных элементов из-за инверсии неиспользуемых старших битов последнего блока памяти.
- Настроена отказоустойчивая обработка потокового ввода: программа корректно реагирует на некорректные данные и выход за границы диапазона, генерируя стандартные исключения C++ (std::invalid_argument и std::out_of_range).

Таким образом, все задачи, поставленные в начале лабораторной работы, выполнены.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Сайт cppreference: https://en.cppreference.com/
2. Информация о битовых полях: https://en.wikipedia.org/wiki/Bit_field

Приложение

Работа автоматических Google Tests:

```
Running main() from D:\a\_work\1\s\googletest\googletest\src\gtest_main.cc
[==========] Running 18 tests from 3 test cases.
[----------] Global test environment set-up.
[----------] 7 tests from TBFTest
[ RUN      ] TBFTest.GetLenBit
[       OK ] TBFTest.GetLenBit (1 ms)
[ RUN      ] TBFTest.SetGet
[       OK ] TBFTest.SetGet (0 ms)
[ RUN      ] TBFTest.ClrBit
[       OK ] TBFTest.ClrBit (0 ms)
[ RUN      ] TBFTest.Copy
[       OK ] TBFTest.Copy (0 ms)
[ RUN      ] TBFTest.OperPris
[       OK ] TBFTest.OperPris (0 ms)
[ RUN      ] TBFTest.OAN
[       OK ] TBFTest.OAN (0 ms)
[ RUN      ] TBFTest.OperSrav
[       OK ] TBFTest.OperSrav (0 ms)
[----------] 7 tests from TBFTest (4 ms total)

[----------] 8 tests from TSTest
[ RUN      ] TSTest.ConstrBOper
[       OK ] TSTest.ConstrBOper (0 ms)
[ RUN      ] TSTest.CConstrPris
[       OK ] TSTest.CConstrPris (0 ms)
[ RUN      ] TSTest.OperPM
[       OK ] TSTest.OperPM (0 ms)
[ RUN      ] TSTest.OperM
[       OK ] TSTest.OperM (0 ms)
[ RUN      ] TSTest.OperSrav
[       OK ] TSTest.OperSrav (0 ms)
[ RUN      ] TSTest.ConBF
[       OK ] TSTest.ConBF (0 ms)
[ RUN      ] TSTest.Univers
[       OK ] TSTest.Univers (0 ms)
[ RUN      ] TSTest.UniversR
[       OK ] TSTest.UniversR (0 ms)
[----------] 8 tests from TSTest (4 ms total)

[----------] 3 tests from TSEx
[ RUN      ] TSEx.InpInv
[       OK ] TSEx.InpInv (0 ms)
[ RUN      ] TSEx.InpThr
[       OK ] TSEx.InpThr (0 ms)
[ RUN      ] TSEx.InpStop
[       OK ] TSEx.InpStop (0 ms)
[----------] 3 tests from TSEx (2 ms total)

[----------] Global test environment tear-down
[==========] 18 tests from 3 test cases ran. (10 ms total)
[  PASSED  ] 18 tests.
```